

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ імені Тараса Шевченка
ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ
Кафедра програмних систем і технологій

Дисципліна
« Комп'ютерна графіка і візуалізація »

Лабораторна робота № 2

Виконав:	Дацишин Володимир	Перевірила:	Жабська Єлизавета Олегівна
Група	ІПЗ-23		
Форма навчання	денна	Оцінка	
Спеціальність	121		
2026			

Тема: додавання прозорості, кадрування та змінення контрастності зображень.

Мета: ознайомитися з методами обробки зображень для створення ефектів прозорості, кадрування та покращення якості за допомогою збільшення контрастності; отримати практичний досвід у розробці функцій для роботи з шарами, обробки окремих областей зображення та оптимізації його візуального вигляду; реалізувати програму, яка забезпечує інтерактивний інтерфейс для виконання цих задач, та дослідити вплив обробки на кінцевий результат.

Завдання для виконання:

- **1. Прозорість зображення:** Реалізувати можливість додавання прозорості до зображення. Програма повинна дозволяти змінювати прозорість шару на зображенні для створення ефектів накладання (alpha blending) та вилучати колірне тло.
- **2. Кадрування зображення:** Реалізувати можливість для користувача розбити зображення на кількість частин, обрану користувачем; вибрати область зображення, вилучити її із зображення (cutout/mask) або вилучити всі частини зображення за межами вибраної області (crop/keep region).
- **3. Збільшення контрастності зображення:** Створити функцію для зміни контрастності зображення за теоретичною формулою: $NewY := K * (OldY - AveY) + AveY$, де AveY — середня яскравість каналу (двопрохідний алгоритм із кліпінгом у $[0..255]$). Дослідити випадки підвищення ($K > 1$) та зниження ($K < 1$) контрастності.

ХІД РОБОТИ

1. Реалізація методів роботи з прозорістю та шарами

Відповідно до теоретичних відомостей, прозорість зображення реалізується за допомогою альфа-каналу, значення якого варіюються в діапазоні 0..255 (де 0 відповідає повній прозорості, а 255 - повній непрозорості). У роботі реалізовано комплекс функцій для обробки прозорості:

- **• set_image_transparency(input_path, output_path, alpha, is_percent):** Змінює глобальний рівень прозорості зображення. Якщо вхідне зображення має формат RGB, воно автоматично конвертується в RGBA. Для встановлення альфа-каналу використовується оптимізований метод Image.putalpha(alpha_val), де значення альфа може задаватися як у діапазоні 0..255, так і у відсотках 0..100%.
- **• overlay_layer(base_path, overlay_path, output_path, position, overlay_opacity):** Виконує накладання напівпрозорого шару (overlay) на базове зображення (base). Реалізовано алгоритм змішування шарів (Alpha Blending) за допомогою Image.alpha_composite(). Перед накладанням шар масштабується за непрозорістю через NumPy і позиціонується на прозорому полотні розміру базового кадру.
- **• make_color_transparent(input_path, output_path, target_rgb, tolerance):** Реалізує техніку Chroma Key (вилучення однорідного тла). За

допомогою векторизованих операцій NumPy обчислюється евклідова/манхеттенська різниця кольорів $\text{diff} \leq \text{tolerance}$, і для всіх відповідних пікселів альфа-канал обнуляється ($\text{arr}[\text{mask}, 3] = 0$).

2. Реалізація методів кадрування та розбиття зображення

Кадрування дозволяє акцентувати увагу на композиційному центрі, усунути небажані елементи або адаптувати розміри зображення під задані пропорції. Розроблено функції для всіх сценаріїв, зазначених у завданні:

- **split_image(input_path, output_dir, rows, cols, prefix):** Розбиває зображення на N рівних частин за координатною сіткою Rows x Cols (наприклад, 1x3 за правилом третин або матриця 2x2). Межі сегментів обчислюються без втрати граничних пікселів, а кожен фрагмент зберігається в окремий файл за допомогою методу Image.crop((left, top, right, bottom)).
- **crop_keep_region(input_path, output_path, box):** Кадрування зі збереженням вибраної прямокутної області [x1, y1, x2, y2] та вилученням усіх частин зображення за її межами. Здійснює попередню нормалізацію координат до меж зображення.
- **crop_remove_region(input_path, output_path, box, fill_mode, fill_color):** Вилучення вибраної області із зображення. Залежно від обраного режиму fill_mode, виділена прямокутна зона або робиться прозорою (встановлюється $\text{arr}[y1:y2, x1:x2, 3] = 0$ у форматі RGBA), або зафарбовується solid-кольором fill_color.
- **center_crop_aspect_ratio(input_path, output_path, aspect_w, aspect_h):** Кадрування з фіксованим центром із дотриманням популярних пропорцій (16:9, 4:3, 1:1) без деформації геометрії.

3. Реалізація математичного алгоритму зміни контрастності

Зміна контрастності без спотворення середнього рівня яскравості по зображенню описується двохпрохідним алгоритмом:

$$\text{NewY} := K * (\text{OldY} - \text{AveY}) + \text{AveY}$$

де K - коефіцієнт контрастності ($K > 1$ - збільшення, $0 < K < 1$ - зменшення), OldY - поточне значення колірному каналу, а AveY — середнє значення того ж каналу по всій площі зображення. При виході значень за межі діапазону [0..255] застосовується обов'язкова корекція (кліпінг).

- **adjust_contrast(input_path, output_path, k):** Поканально обчислює $\text{AveY} = \text{np.mean}(\text{arr[:, :, c]})$ для кожного каналу R, G, B, виконує масштабування відхилень з коефіцієнтом K та накладає кліпінг $\text{np.clip}(..., 0, 255)$. Додатково розраховує та повертає метрики якості: середню яскравість AveY до і після (для підтвердження збереження балансу) та стандартне відхилення σ (міра контрасту).
- **auto_contrast_stretch(input_path, output_path, percentile_cutoff):** Алгоритм лінійного розтягування контрастності динамічного діапазону з відсіканням крайніх квантилів (percentile stretch).

ВИКОРИСТАНІ ФУНКЦІЇ

1. PIL.Image

- **Image.open(path)** - відкриття файлу та зчитування даних
- **Image.putalpha(val)** - додавання або перезапис значень альфа-каналу
- **Image.alpha_composite(base, overlay)** - композиція двох RGBA шарів з урахуванням прозорості
- **Image.crop(box)** - вирізання прямокутного фрагмента зображення
- **Image.fromarray(arr)** - перетворення багатовимірного масиву NumPy назад у графічний об'єкт

2. NumPy

- **np.mean(arr, axis=...)** - обчислення середньої яскравості AveY каналів зображення
- **np.std(arr)** - обчислення середньоквадратичного відхилення (сігма) (числова міра контрастності)
- **np.clip(arr, 0, 255)** - обмеження числових значень пікселів у допустимий діапазон байта

ЛІСТИНГ КОДУ ПРОГРАМИ З КОМЕНТАРЯМИ

```
from __future__ import annotations
import os
import sys
import numpy as np
from PIL import Image

# HELPER FUNCTIONS
def save_image(img: Image.Image, output_path: str) -> None:
    """Saves an image, creating parent directories and converting
    RGBA to RGB for JPEG."""
    os.makedirs(os.path.dirname(output_path) or ".", exist_ok=True)
    ext = os.path.splitext(output_path)[1].lower()
    if ext in (".jpg", ".jpeg") and img.mode in ("RGBA", "LA",
    "P"):
        bg = Image.new("RGB", img.size, (255, 255, 255))
        bg.paste(img, mask=img.split()[-1])
        img = bg
    img.save(output_path)
    size_kb = os.path.getsize(output_path) / 1024.0
    print(f"Saved: {output_path} ({img.width}x{img.height},
    {size_kb:.2f} KB)")
```

```

# TASK 1: IMAGE TRANSPARENCY AND LAYERS

def set_transparency(input_path: str, output_path: str, alpha: int
= 128) -> None:
    """
    Modifies image transparency using the alpha channel.
    alpha: value from 0 (fully transparent) to 255 (fully opaque).
    """
    with Image.open(input_path) as img:
        rgba = img.convert("RGBA")
        rgba.putalpha(int(np.clip(alpha, 0, 255)))
        save_image(rgba, output_path)
        print(f"    • Alpha channel set to: {alpha}/255 ({alpha / 255
* 100:.1f}%)")

def make_color_transparent(input_path: str, output_path: str,
target_rgb=(240, 240, 240), tolerance: int = 15) -> None:
    """Removes a solid background color (Chroma Key), converting it
to transparent."""
    with Image.open(input_path) as img:
        arr = np.array(img.convert("RGBA"), dtype=np.int32)
        diff = np.abs(arr[:, :, :3] - np.array(target_rgb,
dtype=np.int32))
        mask = np.all(diff <= tolerance, axis=-1)
        arr[mask, 3] = 0 # Zero out alpha channel for background
pixels
        res = Image.fromarray(arr.astype(np.uint8), mode="RGBA")
        save_image(res, output_path)
        print(f"    • Made {np.count_nonzero(mask):,} pixels of color
RGB{target_rgb} transparent")

def overlay_layer(base_path: str, overlay_path: str, output_path:
str, pos=(0, 0), opacity: float = 1.0) -> None:
    """
    Overlays a semi-transparent layer onto a base image with
adjustable opacity and position.
    """
    with Image.open(base_path) as base, Image.open(overlay_path) as
over:
        base = base.convert("RGBA")
        over = over.convert("RGBA")
        if opacity < 1.0:
            arr = np.array(over, dtype=np.float32)
            arr[:, :, 3] = np.clip(arr[:, :, 3] * opacity, 0, 255)

```

```

        over = Image.fromarray(arr.astype(np.uint8),
mode="RGBA")

        canvas = Image.new("RGBA", base.size, (0, 0, 0, 0))
        canvas.paste(over, pos)
        result = Image.alpha_composite(base, canvas)
        save_image(result, output_path)
        print(f"    • Overlaid {over.size} layer at position {pos}
with opacity {opacity * 100:.0f}%")

# TASK 2: CROPPING AND SPLITTING IMAGES
def split_image(input_path: str, output_dir: str, num_parts: int =
3, orientation: str = "v") -> list[str]:
    """
    Splits an image into a chosen number of equal segments.
    orientation: 'v' - vertical strips (rule of thirds), 'h' -
horizontal strips.
    """
    os.makedirs(output_dir, exist_ok=True)
    generated = []
    with Image.open(input_path) as img:
        w, h = img.size
        ext = os.path.splitext(input_path)[1] or ".png"
        for i in range(num_parts):
            if orientation == "v":
                left = int(round(i * w / num_parts))
                right = int(round((i + 1) * w / num_parts))
                box = (left, 0, right, h)
            else:
                top = int(round(i * h / num_parts))
                bottom = int(round((i + 1) * h / num_parts))
                box = (0, top, w, bottom)

            part = img.crop(box)
            out_file = os.path.join(output_dir,
f"third_r1_c{i+1}_{i+1}{ext}")
            save_image(part, out_file)
            generated.append(out_file)

        print(f"    • Image {w}x{h} split into {num_parts} parts in
'{output_dir}'")
        return generated

def crop_keep_region(input_path: str, output_path: str, box:
tuple[int, int, int, int]) -> None:

```

```

    """Crops the image to keep only the selected bounding box,
discarding outside areas."""
    with Image.open(input_path) as img:
        x1, y1, x2, y2 = box
        cropped = img.crop((x1, y1, x2, y2))
        save_image(cropped, output_path)
        print(f"    • Kept region {box}: new dimensions
{cropped.width}x{cropped.height}")

def crop_remove_region(input_path: str, output_path: str, box:
tuple[int, int, int, int], fill_color=None) -> None:
    """
    Removes the selected bounding box: makes it transparent or fills
with solid color.
    """
    with Image.open(input_path) as img:
        x1, y1, x2, y2 = box
        if fill_color is None: # Transparent cutout (Alpha = 0)
            res = img.convert("RGBA")
            arr = np.array(res)
            arr[y1:y2, x1:x2, 3] = 0
            res = Image.fromarray(arr, mode="RGBA")
        else: # Solid color fill
            res = img.convert("RGB")
            arr = np.array(res)
            arr[y1:y2, x1:x2, :3] = fill_color
            res = Image.fromarray(arr, mode="RGB")
        save_image(res, output_path)
        print(f"    • Removed rectangular region {box} ({x2 - x1}x{y2
- y1} px)")

# TASK 3: CONTRAST ADJUSTMENT
def adjust_contrast(input_path: str, output_path: str, k: float =
1.5) -> None:
    """
    Adjusts image contrast using the formula:
        NewY := K * (OldY - AveY) + AveY
    where AveY is the mean channel brightness, K is the contrast
factor.
    """
    with Image.open(input_path) as img:
        has_alpha = (img.mode == "RGBA")
        work_img = img.convert("RGBA" if has_alpha else "RGB")

```

```

arr = np.array(work_img, dtype=np.float32)

mean_before = float(np.mean(arr[:, :, :3]))
std_before = float(np.std(arr[:, :, :3]))

# Two-pass algorithm: compute AveY per channel and clip
NewY to [0..255]
    for c in range(3):
        ave_y = float(np.mean(arr[:, :, c]))
        arr[:, :, c] = np.clip(k * (arr[:, :, c] - ave_y) +
ave_y, 0, 255)

mean_after = float(np.mean(arr[:, :, :3]))
std_after = float(np.std(arr[:, :, :3]))

mode = "RGBA" if has_alpha else "RGB"
result = Image.fromarray(arr.astype(np.uint8), mode=mode)
save_image(result, output_path)
print(f"    • Factor K = {k:.2f} | Contrast  $\sigma$ :
{std_before:.1f} -> {std_after:.1f} "
      f"(({std_after - std_before} / std_before *
100:+.1f}%) | AveY: {mean_before:.1f} -> {mean_after:.1f}")

# INTERACTIVE CLI MENU
def main() -> None:
    while True:
        print("\n" + "=" * 55)
        print("    LABORATORY WORK #2: IMAGE PROCESSING")
        print("=" * 55)
        print("1. Change transparency (putalpha)")
        print("2. Remove background color (Chroma Key)")
        print("3. Overlay semi-transparent layer")
        print("4. Split image into N parts")
        print("5. Crop: keep selected region")
        print("6. Crop: remove selected region
(transparent/color)")
        print("7. Adjust contrast (NewY formula)")
        print("8. Exit")
        print("-" * 55)

        try:
            choice = input("Select an option (1-8): ").strip()
        except (EOFError, KeyboardInterrupt):

```

```

        print("\nExiting program.")
        break

    if choice == "1":
        inp = input("Enter input image path: ").strip()
        out = input("Enter output image path: ").strip()
        val = int(input("Enter alpha value (0-255): ").strip())
        set_transparency(inp, out, val)

    elif choice == "2":
        inp = input("Enter input image path: ").strip()
        out = input("Enter output image path: ").strip()
        make_color_transparent(inp, out)

    elif choice == "3":
        base = input("Enter base image path: ").strip()
        over = input("Enter overlay image path: ").strip()
        out = input("Enter output image path: ").strip()
        pos_x = int(input("Enter X coordinate for overlay: ").strip())
        pos_y = int(input("Enter Y coordinate for overlay: ").strip())
        opac = float(input("Enter overlay opacity (0.0..1.0): ").strip())
        overlay_layer(base, over, out, pos=(pos_x, pos_y),
opacity=opac)

    elif choice == "4":
        inp = input("Enter input image path: ").strip()
        out_dir = input("Enter output directory for parts: ").strip()
        n = int(input("Enter number of parts: ").strip())
        split_image(inp, out_dir, num_parts=n)

    elif choice == "5":
        inp = input("Enter input image path: ").strip()
        out = input("Enter output image path: ").strip()
        box_str = input("Enter coordinates X1,Y1,X2,Y2 (comma-separated): ").strip()
        box = tuple(map(int, box_str.split(",")))
        crop_keep_region(inp, out, box)

    elif choice == "6":
        inp = input("Enter input image path: ").strip()

```

```

        out = input("Enter output image path: ").strip()
        box_str = input("Enter coordinates X1,Y1,X2,Y2
(comma-separated): ").strip()
        mode = input("Select removal mode (1 - transparent
window, 2 - white fill): ").strip()
        box = tuple(map(int, box_str.split(",")))
        crop_remove_region(inp, out, box, fill_color=(255, 255,
255) if mode == "2" else None)

    elif choice == "7":
        inp = input("Enter input image path: ").strip()
        out = input("Enter output image path: ").strip()
        k = float(input("Enter contrast factor K (>1 increase,
<1 decrease): ").strip())
        adjust_contrast(inp, out, k)

    elif choice == "8":
        print("\nProgram finished!")
        break

if __name__ == "__main__":
    main()

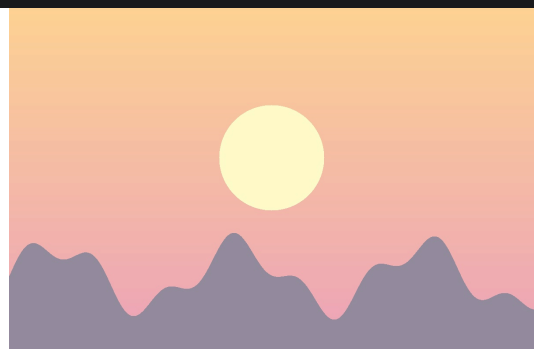
```

РЕЗУЛЬТАТИ ВИКОНАННЯ

```

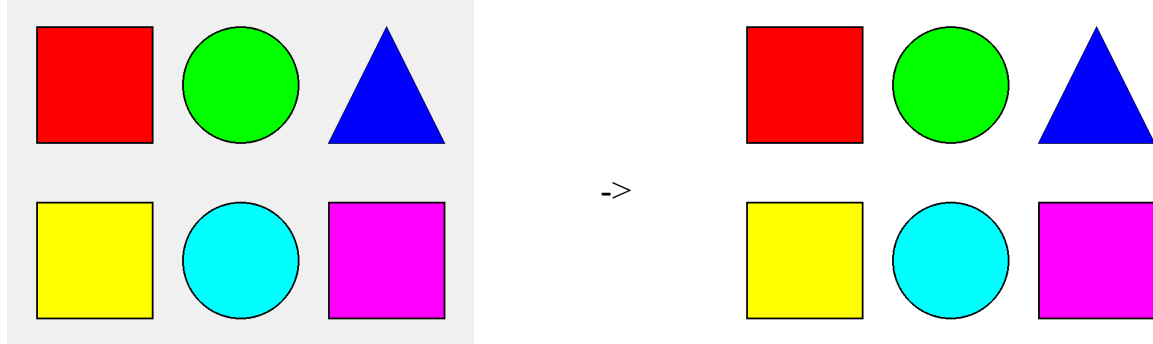
=====
LABORATORY WORK #2: IMAGE PROCESSING
=====
1. Change transparency (putalpha)
2. Remove background color (Chroma Key)
3. Overlay semi-transparent layer
4. Split image into N parts
5. Crop: keep selected region
6. Crop: remove selected region (transparent/color)
7. Adjust contrast (NewY formula)
8. Exit
-----
Select an option (1-8): 1
Enter input image path: test_images/test_landscape.jpg
Enter output image path: output_transparency/landscape_alpha_50pct.png
Enter alpha value (0-255): 128
Saved: output_transparency/landscape_alpha_50pct.png (1200x800, 109.82 KB)
  • Alpha channel set to: 128/255 (50.2%)

```

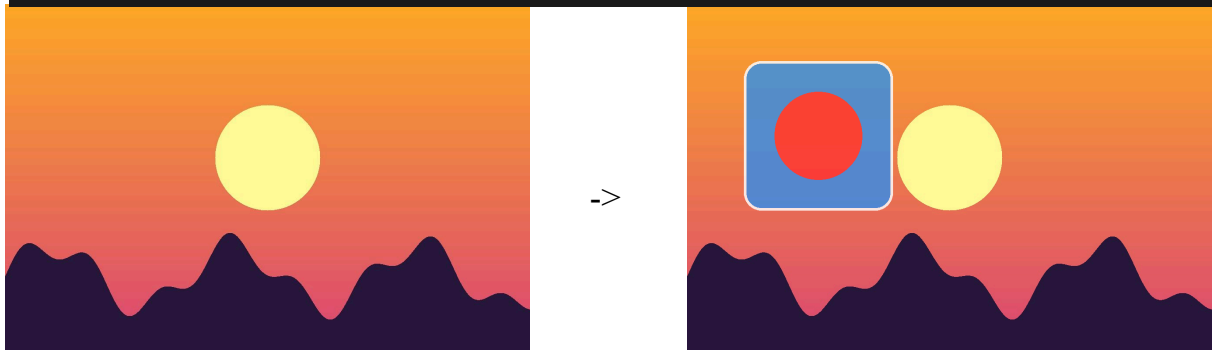


->

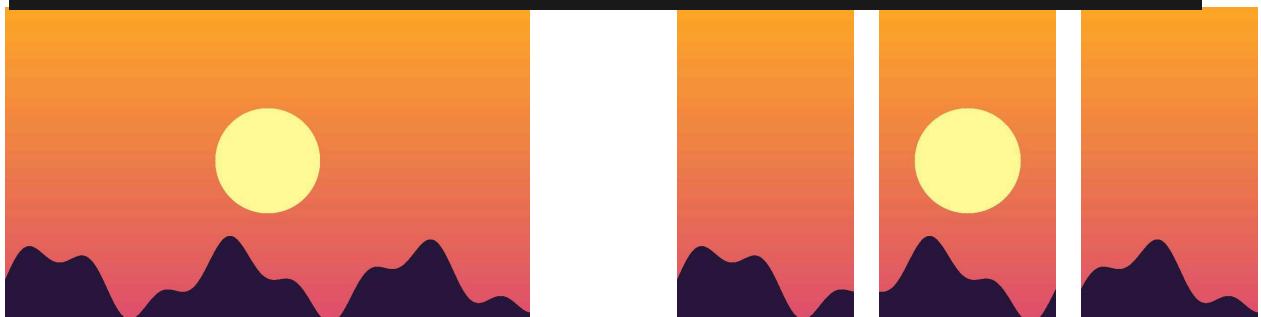
```
-----
Select an option (1-8): 2
Enter input image path: test_images/test_shapes.png
Enter output image path: output_transparency/shapes_transparent_bg.png
Saved: output_transparency/shapes_transparent_bg.png (800x600, 6.02 KB)
  • Made 275,118 pixels of color RGB(240, 240, 240) transparent
```



```
-----
Select an option (1-8): 3
Enter base image path: test_images/test_landscape.jpg
Enter overlay image path: test_images/test_transparent.png
Enter output image path: output_transparency/landscape_with_overlay.png
Enter X coordinate for overlay: 100
Enter Y coordinate for overlay: 100
Enter overlay opacity (0.0..1.0): 0.85
Saved: output_transparency/landscape_with_overlay.png (1200x800, 112.29 KB)
  • Overlaid (400, 400) layer at position (100, 100) with opacity 85%
```

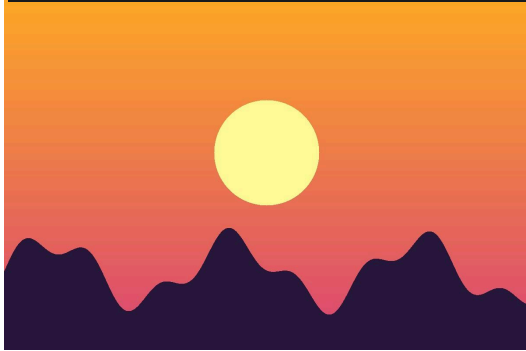


```
-----
Select an option (1-8): 4
Enter input image path: test_images/test_landscape.jpg
Enter output directory for parts: output_cropping/split_3parts
Enter number of parts: 3
Saved: output_cropping/split_3parts\third_r1_c1_1.jpg (400x800, 8.09 KB)
Saved: output_cropping/split_3parts\third_r1_c2_2.jpg (400x800, 10.29 KB)
Saved: output_cropping/split_3parts\third_r1_c3_3.jpg (400x800, 8.00 KB)
  • Image 1200x800 split into 3 parts in 'output_cropping/split_3parts'
```



->

```
-----  
Select an option (1-8): 5  
Enter input image path: test_images/test_landscape.jpg  
Enter output image path: output_cropping/cropped_focus_center.jpg  
Enter coordinates X1,Y1,X2,Y2 (comma-separated): 300,200,900,600  
Saved: output_cropping/cropped_focus_center.jpg (600x400, 7.97 KB)  
  • Kept region (300, 200, 900, 600): new dimensions 600x400
```



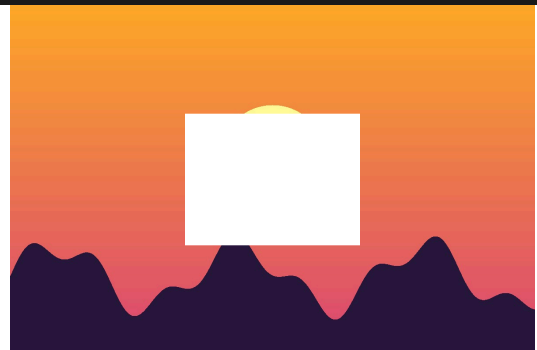
->



```
-----  
Select an option (1-8): 6  
Enter input image path: test_images/test_landscape.jpg  
Enter output image path: output_cropping/landscape_removed_center_transparent.png  
Enter coordinates X1,Y1,X2,Y2 (comma-separated): 400,250,800,550  
Select removal mode (1 - transparent window, 2 - white fill): 1  
Saved: output_cropping/landscape_removed_center_transparent.png (1200x800, 109.51 KB)  
  • Removed rectangular region (400, 250, 800, 550) (400x300 px)
```



->



```
-----  
Select an option (1-8): 7  
Enter input image path: test_images/test_landscape.jpg  
Enter output image path: output_contrast/landscape_contrast_high_k1_6.jpg  
Enter contrast factor K (>1 increase, <1 decrease): 1.6  
Saved: output_contrast/landscape_contrast_high_k1_6.jpg (1200x800, 28.74 KB)  
  • Factor K = 1.60 | Contrast  $\sigma$ : 79.9 -> 93.6 (+17.2%) | AveY: 126.7 -> 127.7
```



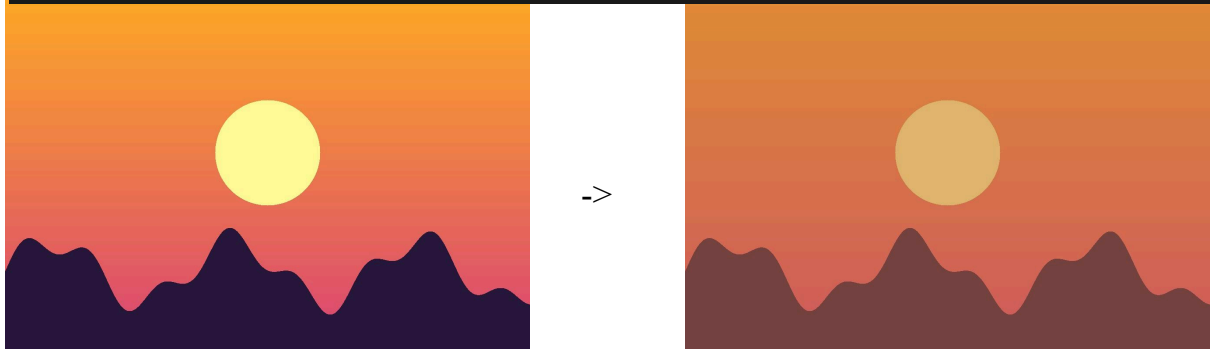
->



```

-----
Select an option (1-8): 7
Enter input image path: test_images/test_landscape.jpg
Enter output image path: output_contrast/landscape_contrast_low_k0_5.jpg
Enter contrast factor K (>1 increase, <1 decrease): 0.5
Saved: output_contrast/landscape_contrast_low_k0_5.jpg (1200x800, 21.23 KB)
  • Factor K = 0.50 | Contrast  $\sigma$ : 79.9 -> 60.0 (-24.8%) | AveY: 126.7 -> 126.7

```



Висновок: Під час виконання лабораторної роботи було успішно досягнуто поставленої мети: ознайомлено з основними методами цифрової обробки зображень для створення ефектів прозорості, кадрування та оптимізації якості за допомогою регулювання контрастності. У процесі виконання завдань було набуто практичного досвіду розробки алгоритмів і функцій для роботи з шарами, фрагментами зображень та їх візуальними параметрами: опановано принципи функціонування альфа-каналу: реалізовано керування рівнем прозорості зображення, змішування та накладання шарів, а також техніку вилучення однорідного колірному фону; реалізовано методи просторового кадрування: поділ зображення на задану кількість рівних частин, виділення та збереження цільової області із відсіканням зайвих елементів, а також вилучення обраної ділянки із зображення; досліджено алгоритм корекції контрастності, що дозволяє динамічно підсилювати або пом'якшувати перепади між світлими та темними ділянками, зберігаючи при цьому загальну середню яскравість зображення; розроблено програму із зручним інтерфейсом взаємодії з користувачем, яка дозволяє гнучко застосовувати розглянуті операції та наочно оцінювати вплив кожного етапу обробки на кінцевий візуальний результат.

ВІДПОВІДІ НА ПИТАННЯ ДЛЯ САМОКОНТРОЛЮ

1. Що таке прозорість у зображеннях, і як вона реалізується технічно?

Прозорість у цифровій графіці — це характеристика пікселя, що визначає ступінь його непрозорості (opacity) та здатність пропускати крізь себе пікселі нижчих шарів. Технічно прозорість реалізується за допомогою додаткового альфа-каналу (A в моделі RGBA), де кожному пікселю виділяється додатковий байт (0..255). Значення 0 відповідає абсолютній прозорості, а 255 — повній непрозорості.

2. Які формати зображень підтримують прозорість, і які особливості їх використання?

Прозорість підтримують такі популярні формати:

- PNG (Portable Network Graphics) — підтримує повноцінний 8-бітний альфа-канал (256 рівнів прозорості) зі стисненням без втрат;
 - WebP — сучасний формат, що підтримує 8-бітний альфа-канал як для стиснення без втрат, так і з втратами;
 - TIFF — професійний формат для поліграфії, що зберігає численні кольорні та альфа-канали;
 - GIF — підтримує лише бінарну 1-бітну прозорість (піксель або повністю прозорий, або повністю непрозорий) в обмеженій палітрі 256 кольорів.
- Формати JPEG та BMP (у стандартній специфікації) прозорість не підтримують.

3. Як прозорість впливає на загальний вигляд зображення при накладанні шарів?

Прозорість забезпечує безшовну інтеграцію графічних об'єктів без грубих прямокутних меж та небажаних фонів. При накладанні шарів використовується формула Alpha Blending: $C = C_top * alpha + C_bottom * (1 - alpha)$, що створює плавні переходи кольорів, реалістичні напівпрозорі тіні, відблиски та ефекти глибини сцени.

4. Що таке альфа-канал, і яку роль він відіграє у реалізації прозорості?

Альфа-канал — це окрема матриця градацій прозорості однакової розмірності з кольірними матрицями (R, G, B). Він виступає ваговим коефіцієнтом для розрахунку внеску поточного шару у фінальний рендеринг кожного пікселя композиції.

5. Що таке кадрування зображення, і для яких завдань його використовують?

Кадрування (cropping) — це процес виділення та відсікання частини зображення з метою поліпшення його композиції, акцентування уваги на головному смислового об'єкті, видалення небажаних периферійних деталей, а також зміни фізичних розмірів або співвідношення сторін під вимоги друку чи веб-платформ.

7. Як здійснюється поділ зображення на частини в межах програми? Які алгоритми для цього використовуються?

Поділ зображення здійснюється шляхом розрахунку прямокутних координат (Bounding Boxes) для кожного сегмента: $\text{left} = \text{round}(c * W / \text{Cols})$, $\text{right} = \text{round}((c + 1) * W / \text{Cols})$, $\text{top} = \text{round}(r * H / \text{Rows})$, $\text{bottom} = \text{round}((r + 1) * H / \text{Rows})$. Після цього кожен сегмент вирізається за допомогою зрізів масивів NumPy (`arr[top:bottom, left:right]`) або методу PIL `Image.crop()` і зберігається у файл. Такий алгоритм гарантує точне збереження всіх пікселів без перекриттів чи втрат на стиках.

8. Які методи можна застосувати для вибору області зображення?

Для вибору області застосовуються:

- 1) Прямокутний вибір за абсолютними координатами (x_1, y_1, x_2, y_2);
- 2) Кадрування за фіксованими пропорціями (Aspect Ratio 16:9, 4:3, 1:1) відносно центру;
- 3) Правило третин (виділення вузлових ліній та квадрантів сітки 3x3);
- 4) Масковий вибір (Masking) за допомогою альфа-каналу або бінарної маски довільної форми.

9. Що таке контрастність зображення, і чому її важливо контролювати?

Контрастність — це ступінь візуальної різниці між найбільш світлими та найбільш темними ділянками зображення (динамічний діапазон та крутизна перепадів яскравості). Контроль контрастності є критично важливим, оскільки надмірно низький контраст робить зображення млявим, тьмяним і нерозбірливим, а надмірний контраст веде до кліпінгу — безповоротної втрати деталей у тінях (*underexposure*) та світлих зонах (*overexposure*).

10. Як збільшення контрастності впливає на візуальне сприйняття зображення?

Збільшення контрастності робить зображення більш чітким, виразним, динамічним і візуально привабливим; кольори сприймаються як більш насичені, а грані та контури об'єктів стають помітнішими.

11. Які математичні операції використовуються для підвищення контрастності?

Для регулювання контрастності використовується лінійна трансформація відхилення пікселя від середнього значення:

$$\text{NewY} = \text{clip}(K * (\text{OldY} - \text{AveY}) + \text{AveY}, 0, 255)$$

де обчислення середнього AveY вимагає першого проходу сумування, множення на $K > 1$ збільшує розмах амплітуди, додавання AveY центрує розподіл навколо первинної яскравості, а функція `clip` виконує сатурацію значень у межах 8-бітного діапазону.